

Εργαστηριακές Ασκήσεις Ψηφιακά Συστήματα HW σε Χαμηλά Επίπεδα Λογικής I

Ονοματεπώνυμο: Ηλίας Σίμος

Άσκηση 1

Ο σκοπός της άσκησης, είναι η δημιουργία μιας ALU (Arithmetic Logic Unit), η οποία εκτελεί αριθμητικές πράξεις όπως πρόσθεση και πολλαπλασιασμό και λογικές πράξεις όπως AND ή OR. Αρχικά, δημιουργήσαμε το module της ALU με θύρες εισόδου και εξόδου όπως ζητούνται στην εκφώνηση. Στη συνέχεια, μέσω της εντολής parameter καθορίσαμε τις σταθερές για το alu operation ως εξής:

```
parameter[3:0] ALUOP_AND = 4'b1000;
parameter[3:0] ALUOP_OR = 4'b1001;
parameter[3:0] ALUOP_NOR = 4'b1010;
parameter[3:0] ALUOP_NAND = 4'b1011;
parameter[3:0] ALUOP_XOR = 4'b1100;
parameter[3:0] ALUOP_ADD = 4'b0100;
parameter[3:0] ALUOP_SUB = 4'b0101;
parameter[3:0] ALUOP_MUL = 4'b0110;
parameter[3:0] ALUOP_LSHIFTR = 4'b0000;
parameter[3:0] ALUOP_LSHIFTL = 4'b0001;
parameter[3:0] ALUOP_ASHIFTR = 4'b0010;
parameter[3:0] ALUOP_ASHIFTL = 4'b0011;
```

Η λογική του κυκλώματος βρίσκεται μέσα σε ένα «always @(*)» block και γίνεται χρήση blocking εντολών, καθώς το κύκλωμα της ALU είναι καθαρά συνδυαστικό. Ο πολυπλέκτης της ALU υλοποιείται με την εντολή case της Verilog. Ανάλογα την τιμή του ALU operation, εκτελείται η αντίστοιχη πράξη μεταξύ των op1 και op2. Αν το αποτέλεσμα της είναι μηδέν, ενεργοποιείται το zero signal. Στην περίπτωση της πρόσθεσης και του πολλαπλασιασμού, κάνουμε και έλεγχο για overflow. Ειδικότερα για τον πολλαπλασιασμό, χρησιμοποιήσαμε μία επιπλέον εσωτερική μεταβλητή για να αποθηκεύσουμε το πλήρες 64-bit γινόμενο (fullprod). Υπερχείλιση συμβαίνει, αν τα ανώτερα bits του πλήρους γινομένου, δεν αποτελούν επέκταση προσήμου του κατώτερου 32-bit αποτελέσματος.

```
---
fullprod = $signed(op1) * $signed(op2);
result = fullprod[31:0];
// overflow detection
ovf = (fullprod[63:32] != {32{fullprod[31]}});
```

Άσκηση 2

Στη συγκεκριμένη άσκηση σχεδιάζουμε ένα κύκλωμα αριθμομηχανής, η οποία ενσωματώνει την ALU της άσκησης 1.

calc.v

Ξεκινώντας τη σχεδίαση, δημιουργήσαμε το module της αριθμομηχανής με εισόδους τα buttons (btnac, btnc, btnl, btnr, btnd), την 16-bit είσοδο sw και το ρολόι. Η έξοδος του κυκλώματος είναι τα 16-bit LED τα οποία θα τυπώνουν το αποτέλεσμα της αριθμομηχανής, σύμφωνα με την εκφώνηση. Δημιουργήσαμε ένα εσωτερικό 32-bit σήμα, με σκοπό να τροφοδοτηθεί σαν ένας από τους δύο operands της ALU. Η τιμή του είναι η 16-bit είσοδος sw με επέκταση προσήμου. Για την επέκταση, χρησιμοποιήσαμε τον τελεστή concatenation της Verilog, προσθέτοντας το MSB της εισόδου sw άλλες 16 φορές.

```
wire signed [31:0] op2 = {{16{sw[15]}}, sw[15:0]};
```

Ύστερα, δημιουργήσαμε έναν 16-bit register ο οποίος έχει τον ρόλο του accumulator. Αρχικοποιήσαμε την ALU, κάνοντας αντιστοίχιση θυρών σύμφωνα με το σχήμα της εκφώνησης. Συγκεκριμένα:

- Η είσοδος op1 είναι η επέκταση προσήμου του accumulator.
- Η είσοδος op2 είναι η επέκταση προσήμου της εισόδου της αριθμομηχανής.
- Οι έξοδοι result, onf, zero, αντιστοιχίζονται σε εσωτερικά nets, τα οποία δημιουργήσαμε.
- Η είσοδος alu_op αντιστοιχίζεται στην έξοδο του calc_enc module, η λειτουργία του οποίου θα εξηγηθεί παρακάτω.

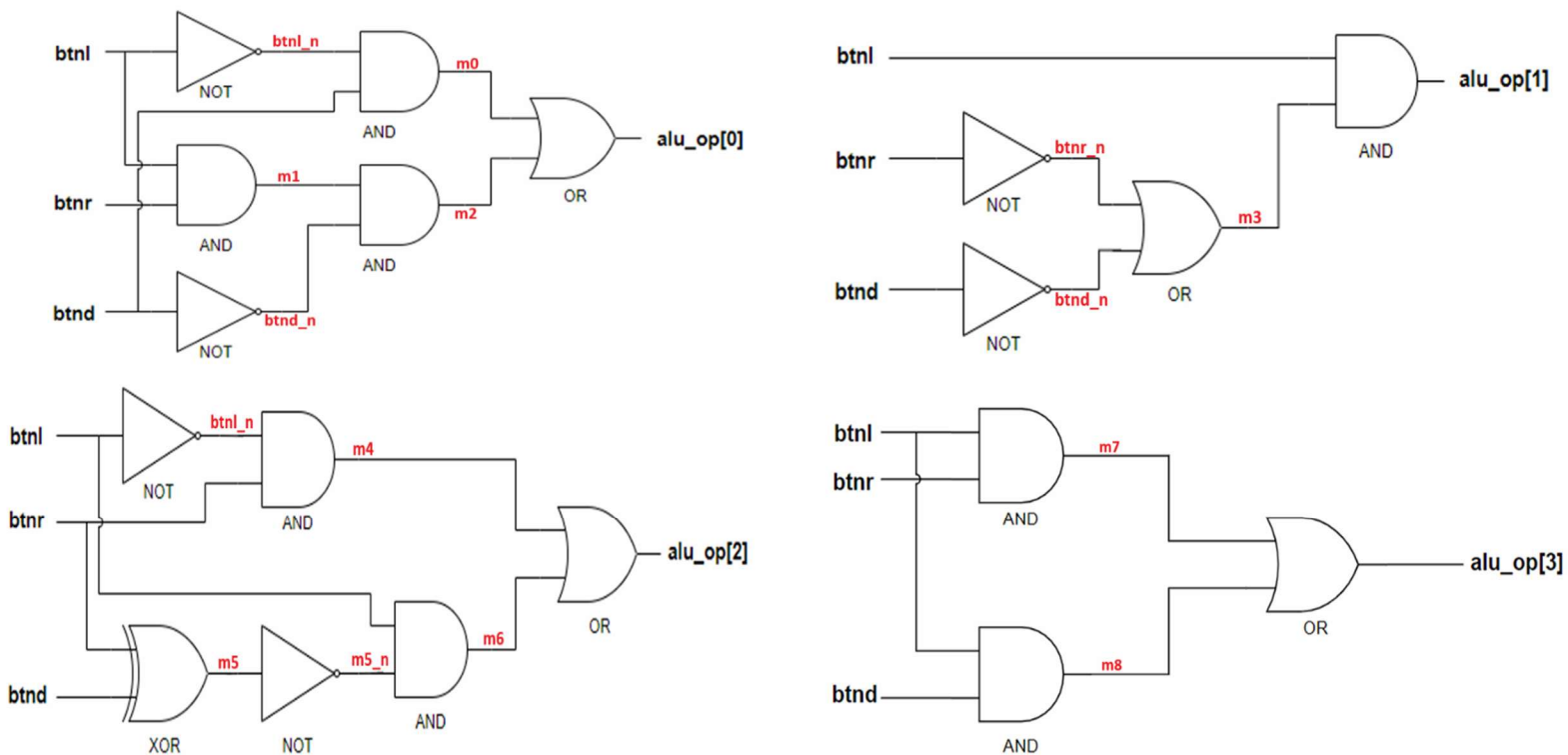
Η λειτουργία του accumulator, η οποία περιγράφεται μέσω ενός always@(posedge clk) block είναι η εξής:

-Ο accumulator μηδενίζεται σύγχρονα όταν btnac = 1, και ενημερώνεται στην ακμή του ρολογιού, όταν btnc = 1.

-Σαν είσοδο παίρνει τα 16 χαμηλότερα bit της εξόδου της ALU και συνδέεται με την έξοδο LED της αριθμομηχανής.

calc_enc.v

Για τον καθορισμό της λειτουργίας της ALU που θα εκτελεστεί, δημιουργήσαμε το module calc_enc, το οποίο παίρνει σαν είσοδο τα 3 buttons της αριθμομηχανής (btnr, btnl, btnd) και εξάγει το operation της ALU. Τα 4 bits του alu_op παράγονται μέσω του συνδυαστικού κυκλώματος της εκφώνησης, το οποίο το μοντελοποιήσαμε με δομική Verilog και χρήση primitives των πυλών not, and, or και xor. Επίσης, προσθέσαμε εσωτερικά nets για τα ενδιάμεσα αποτελέσματα των πυλών, σύμφωνα με τα παρακάτω σχήματα:



Calc_tb.v

Για να ελέγξουμε την ορθή λειτουργία της αριθμομηχανής, δημιουργήσαμε ένα testbench module calc_tb, το οποίο περιέχει ένα instance του calc module

και ένα instance του ρολογιού με περίοδο 10ns. Στο stimulus του testbench, εκτελούμε διαδοχικά δοκιμές σύμφωνα με τον πίνακα της εκφώνησης.

btnl, btnr, btnd (input)	Previous value (acc.)	Switches (input)	Function in ALU	Expected Result
(btnac for reset)	xxxx	xxxx	Reset	0x0
0,1,0	0x0	0x285a	ADD	0x285a
1,1,1	0x285a	0x04c8	XOR	0x2c92
0,0,0	0x2c92	0x0005	Logical Shift Right	0x0164
1,0,1	0x0164	0xa085	NOR	0x5e1a
1,0,0	0x5e1a	0x07fe	MULT	0x13cc
0,0,1	0x13cc	0x0004	Logical Shift Left	0x3cc0
1,1,0	0x3cc0	0xfa65	NAND	0xc7bf
0,1,1	0xc7bf	0xb2e4	SUB	0x14db

Για να το πετύχουμε αυτό, συντάξαμε δύο tasks, τα οποία καλούμε για κάθε ομάδα εισόδων στο initial block του stimulus. Ειδικότερα, το task «calc_operation» θέτει τιμές στα buttons και στην είσοδο της αριθμομηχανής και ενεργοποιεί το btnc ώστε να εκτελεστεί η πράξη. Το task «check_and_display», ελέγχει αν το αποτέλεσμα ταυτίζεται με το αναμενόμενο και το τυπώνει μαζί με τις εισόδους από τις οποίες παρήχθη. Επίσης, τυπώνει PASS αν τα αποτελέσματα ταυτίζονται και FAIL αν τα αποτελέσματα δεν συμφωνούν.

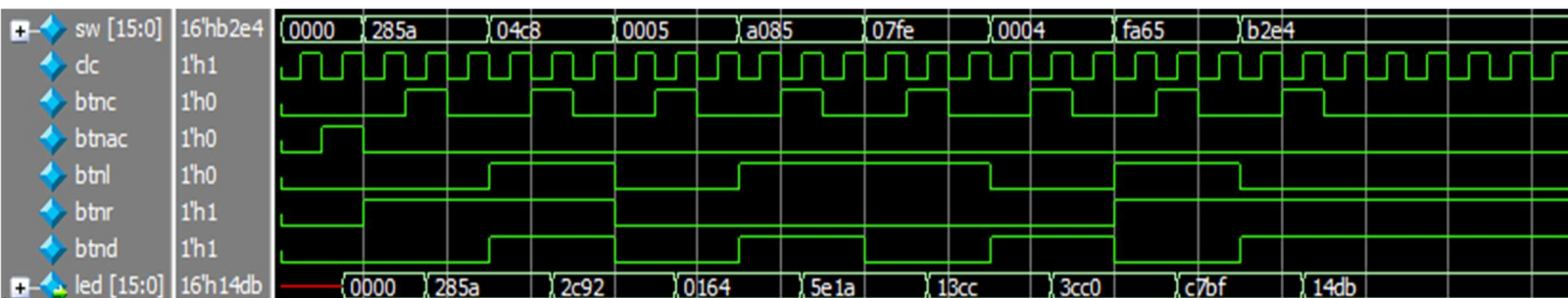
Αφότου συντάξαμε το testbench της αριθμομηχανής, προσομοιώνουμε το κύκλωμα που υλοποιήσαμε. Στην κονσόλα εμφανίζεται το παρακάτω μήνυμα:

```

# Starting Simulation
# btnac = 1 (Reset) : Result: 0x0000, Expected: 0x0000
# btnl|btnr|btnd = 0|1|0
# PASS - Result: 0x285a, Expected: 0x285a
# btnl|btnr|btnd = 1|1|1
# PASS - Result: 0x2c92, Expected: 0x2c92
# btnl|btnr|btnd = 0|0|0
# PASS - Result: 0x0164, Expected: 0x0164
# btnl|btnr|btnd = 1|0|1
# PASS - Result: 0x5e1a, Expected: 0x5e1a
# btnl|btnr|btnd = 1|0|0
# PASS - Result: 0x13cc, Expected: 0x13cc
# btnl|btnr|btnd = 0|0|1
# PASS - Result: 0x3cc0, Expected: 0x3cc0
# btnl|btnr|btnd = 1|1|0
# PASS - Result: 0xc7bf, Expected: 0xc7bf
# btnl|btnr|btnd = 0|1|1
# PASS - Result: 0x14db, Expected: 0x14db
# Simulation Ended

```

Διάγραμμα χρονισμού της αριθμομηχανής



Βλέπουμε ότι, το διάγραμμα χρονισμού συμφωνεί απόλυτα με την αναμενόμενη συμπεριφορά της αριθμομηχανής. Την χρονική στιγμή που «πατιέται» το btnac, δηλαδή μεταβαίνει στην τιμή 1, η έξοδος ενημερώνεται στην ακμή του ρολογιού (λόγω της ύπαρξης του σήματος clk στη sensitivity list του always block και την χρήση non-blocking εντολών) και παίρνει την τιμή της πράξης (που ορίζουν τα buttons btnl, btnr, btnd), ανάμεσα στην είσοδο sw και την προηγούμενη τιμή της εξόδου. Το μήνυμα της κονσόλας, επιβεβαιώνει ότι τα εξαγόμενα αποτελέσματα είναι σωστά.

Άσκηση 3

Στη συγκεκριμένη άσκηση, ζητείται η υλοποίηση ενός αρχείου καταχωρητών (register file), ώστε να χρησιμοποιηθεί για την αποθήκευση των βαρών και των πολώσεων του νευρωνικού δικτύου της επόμενης άσκησης. Αρχικά, δημιουργήσαμε το regfile module με θύρες οι οποίες ακολουθούν αυστηρά τον πίνακα της εκφώνησης. Έπειτα, δημιουργήσαμε έναν πίνακα με όνομα registers ο οποίος υλοποιεί 16 καταχωρητές μεγέθους DATAWIDTH bit , όπου DATAWIDTH παράμετρος που καθορίζει το μέγεθος των λέξεων που αποθηκεύονται στους καταχωρητές, με default τιμή 32 bits. Η λογική του register file είναι η εξής:

- Χρησιμοποιούμε ένα ασύγχρονο active-low reset σήμα, το οποίο μηδενίζει όλους τους καταχωρητές, μέσα σε ένα always @(posedge clk or negedge resetn) block, το οποίο περιγράφει και την λειτουργία του regfile. Η παρουσία του resetn στη sensitivity list του συγκεκριμένου block, το κάνει ασύγχρονο.
- Στο ίδιο block εάν το σήμα write πάρει την τιμή 1, γίνεται εγγραφή (σύγχρονη) των 2 writeData 32-bit λέξεων στους registers που υποδεικνύουν οι 2 4-bit writeReg διευθύνσεις.
- Επιλέξαμε η ανάγνωση στην συγκεκριμένη σχεδίαση του αρχείου καταχωρητών να είναι ασύγχρονη, για αυτό και χρησιμοποιήσαμε ένα παραπάνω always block που δεν περιλαμβάνει το clock στην sensitivity list του.
- Στην περίπτωση που πραγματοποιείται αίτημα εγγραφής και η διεύθυνση εγγραφής ταυτίζεται με την διεύθυνση ανάγνωσης, φροντίσαμε μέσω διαδοχικών if – else statements για κάθε συνδυασμό ReadReg και WriteReg να δίνεται προτεραιότητα στην εγγραφή. Άρα, τα δεδομένα ανάγνωσης είναι ενημερωμένα στην πιο πρόσφατη τιμή τους, δηλαδή στην τιμή που περιλαμβάνει το πιο πρόσφατο αίτημα εγγραφής.

Η ορθή λειτουργία του regfile δεν θα εξεταστεί μέσω ξεχωριστού testbench. Αντί αυτού, το κύκλωμα θα ελεγχθεί, αφού ενσωματωθεί στο νευρωνικό δίκτυο της επόμενης άσκησης.

Άσκηση 4

Στη συγκεκριμένη άσκηση, ζητήθηκε η υλοποίηση ενός νευρωνικού δικτύου (AI accelerator), το οποίο θα αποτελείται από πράξεις αριθμητικής ολίσθησης δεξιά και αριστερά, καθώς και από πράξεις πολλαπλασιασμού και πρόσθεσης.

mac_unit.v

Ο κάθε νευρώνας του δικτύου, εκτελεί μια πράξη MAC (Multiply And Accumulate), δηλαδή πολλαπλασιάζει την είσοδο με ένα βάρος (weight) και στη συνέχεια, προσθέτει και μία πόλωση (bias). Για διευκόλυνση της σχεδίασης, φτιάξαμε ένα mac_unit module, το οποίο έχει θύρες που ταιριάζουν με τον αντίστοιχο πίνακα της εκφώνησης. Η υλοποίηση του mac_unit, περιέχει δύο instances της ALU (σχεδιασμένη στην Άσκηση 1) σε σειρά, κάνοντας κατάλληλη αντιστοίχιση θυρών στα 2 δηλωμένα modules. Για τον σκοπό αυτό, δημιουργήσαμε και ένα εσωτερικό 32-bit σήμα για το ενδιάμεσο αποτέλεσμα. Η πρώτη ALU, θα έχει σαν alu_op τον 3-bit αριθμό που αντιστοιχεί στην πράξη του πολλαπλασιασμού ($op1 * op2$), ενώ η δεύτερη αυτόν που αντιστοιχεί στην πράξη της πρόσθεσης ($(op1 * op2) + op3$). Για καθαρότητα στον κώδικα μας, αντιστοιχήσαμε τους συγκεκριμένους αριθμούς σε ονόματα, χρησιμοποιώντας την εντολή parameter, όπως πράξαμε κατά την σχεδίαση της ALU.

```
parameter[3:0] ALUOP_ADD = 4'b0100;
parameter[3:0] ALUOP_MUL = 4'b0110;
wire signed [31:0] intermiante_result;
```

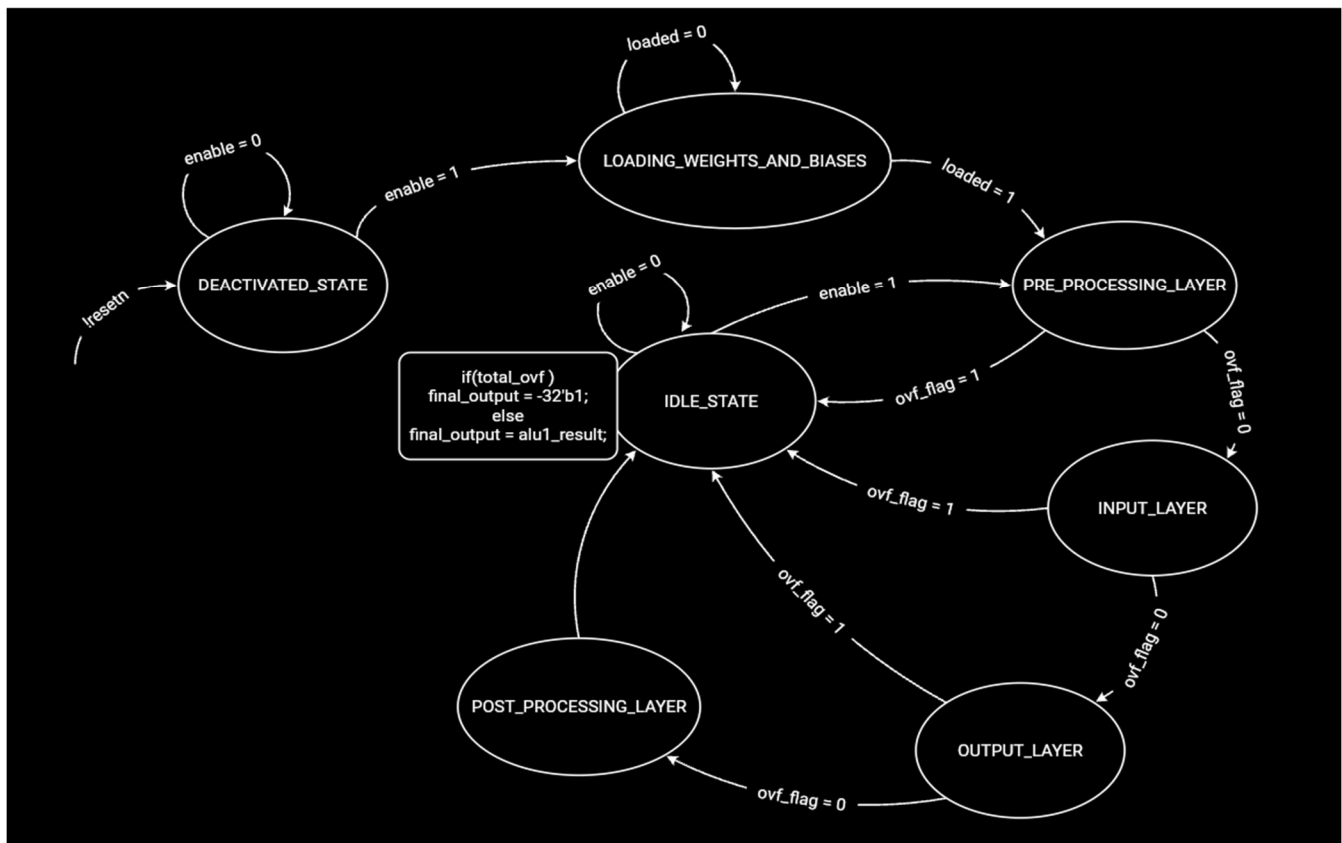
```
alu U1 (.alu_op(ALUOP_MUL), .op1(op1), .op2(op2), .zero(zero_mul), .ovf(ovf_mul), .result(intermiante_result));
alu U2 (.alu_op(ALUOP_ADD), .op1(intermiante_result), .op2(op3), .zero(zero_add), .ovf(ovf_add), .result(total_result));
```

nn.v

Αφού υλοποιήσαμε το mac_unit module που συμπεριλαμβάνεται σε κάθε νευρώνα, προχωρήσαμε στην υλοποίηση του ίδιου του νευρωνικού δικτύου. Αρχικά, δημιουργήσαμε το module του, ονομάζοντας τις θύρες εισόδου και εξόδου βασιζόμενοι στον αντίστοιχο πίνακα της εκφώνησης. Ζητείται να γίνει η υλοποίηση του νευρωνικού δικτύου ως μία μηχανή πεπερασμένης

κατάστασης (Finite State Machine). Οπότε, με χρήση της εντολής parameter κωδικοποιήσαμε τις καταστάσεις του fsm. Επίσης, δημιουργήσαμε instances από δύο ALUs, δύο mac_units, την rom (το module της δόθηκε από την εκφώνηση) και το regfile, στοιχεία τα οποία θα χρησιμοποιηθούν για την υλοποίηση των σταδίων του δικτύου και την φόρτωση και αποθήκευση των βαρών και πολώσεων αντίστοιχα.

Το fsm που επιλέξαμε να υλοποιήσουμε είναι τύπου Moore, καθώς οι τελικές έξοδοι του νευρωνικού δικτύου εξαρτώνται από το αποθηκευμένο στάδιο στο οποίο βρίσκεται η μηχανή (current_state). Το διάγραμμα καταστάσεων του απεικονίζεται παρακάτω.



Μέσω always blocks, υλοποιήσαμε το state memory και το next state logic σύμφωνα με το διάγραμμα καταστάσεων. Η επαναφορά του δικτύου στην DEACTIVATED_STATE γίνεται, χρησιμοποιώντας ένα active low reset σήμα (`!resetn`), το οποίο το συμπεριλαμβάνουμε στην sensitivity list του always block, μιας και ζητείται να είναι ασύγχρονο.

Σημείωση: Δημιουργήσαμε 2 εσωτερικά flags (loaded και onf_flag) ώστε να μπορούμε να γνωρίζουμε την στιγμή που τερματίζει η φόρτωση των βαρών και των πολώσεων από την rom στο regfile (στην κατάσταση `LOADING_WEIGHTS_AND_BIASES`), ώστε το fsm να συνεχίσει σε επόμενο στάδιο, καθώς και όταν συμβαίνει υπερχείλιση σε κάποιο στάδιο του, ώστε να μεταβαίνουμε στην κατάσταση `IDLE`. Το onf_flag χρησιμοποιείται και στην λογική της εξόδου.

Συνεχίζοντας την σχεδίαση, δημιουργήσαμε ένα always block, στο οποίο ανάλογα την τωρινή κατάσταση, θέτουμε τιμές στα εσωτερικά nets του δικτύου ώστε να υλοποιηθεί η λειτουργία του. Στην αρχή, δίνουμε default τιμές στα εσωτερικά nets ώστε να αποφύγουμε τη δημιουργία latches. Αξιοποιούμε την εντολή case της Verilog με όρισμα την τωρινή κατάσταση (current state):

- **DEACTIVATED_STATE:** Κάνουμε reset τους registers στο εσωτερικό regfile.
- **LOADING_WEIGHTS_AND_BIASES:** Μέσω ενός εσωτερικού counter που υλοποιήσαμε, εξάγουμε τις αποθηκευμένες τιμές από την rom και μέσω κατάλληλων αναθέσεων στα writeReg και writeData του regfile τις αποθηκεύουμε σε αυτό. Όταν τελειώσει η διαδικασία, ενεργοποιείται το loaded flag.
- **PRE_PROCESSING_LAYER:** Δίνουμε τιμές στις θύρες των δύο ALU, ώστε να πραγματοποιήσουν ταυτόχρονα αριθμητική ολίσθηση δεξιά στα 2 inputs ($inter1 = inter1 \ggg shift_bias_1$ και $inter2 = inter2 \ggg shift_bias_2$).
- **INPUT_LAYER:** Δίνουμε τιμές στις εσωτερικές θύρες των δύο mac_units ώστε να εκτελέσουν παράλληλα στα ενδιάμεσα αποτελέσματα, τις πράξεις του πολλαπλασιασμού με το αντίστοιχο βάρος και της πρόσθεσης με την αντίστοιχη πόλωση. ($inter_3 = inter_1 * weight_1 + bias_1$ και $inter_4 = inter_2 * weight_2 + bias_2$).

Σημείωση: Είναι σημαντικό να τονίσουμε εδώ ότι, επιλέξαμε να αποθηκεύσουμε όλα τα ενδιάμεσα αποτελέσματα του νευρωνικού δικτύου σε εσωτερικούς registers και όχι στο regfile. Η επιλογή αυτή έγινε γιατί, αν επιλέγαμε την αποθήκευση στο regfile, θα είχαμε πρόβλημα με την παράλληλη επεξεργασία στο στάδιο INPUT_LAYER. Ειδικότερα, κατά τη μετάβαση στο INPUT_LAYER θα έπρεπε να διαβάσουμε τα δύο ενδιάμεσα αποτελέσματα, μαζί με τα δύο βάρη και τις δύο πολώσεις που χρειαζόμαστε για τα mac_units. Άρα, απαιτούνται έξι θύρες ανάγνωσης και, σύμφωνα με τον σχεδιασμό του regfile στην άσκηση 3, έχουμε διαθέσιμες μόνο 4. Επομένως, θα έπρεπε να χωρίσουμε την εκτέλεση των πράξεων, κάτι που επηρεάζει την παραλληλία και την ταχύτητα, καθώς πιθανόν να χρειάζεται περισσότερους κύκλους ρολογιού για να επιτευχθεί.

- **OUTPUT_LAYER:** Συνδυάζουμε τα ενδιάμεσα αποτελέσματα του INPUT_LAYER χρησιμοποιώντας 2 mac_units συνδεδεμένα, με κατάλληλη αντιστοίχιση θυρών, σε σειρά.
$$(inter_5 = inter_3 * weight_3 + inter_4 * weight_4 + bias_3)$$
- **DATA_POST_PROCESSING:** η μία από τις 2 ALU εκτελεί στην έξοδο του OUTPUT_LAYER (inter_5) αριθμητική ολίσθηση αριστερά.
- Στο output logic always block, όταν το fsm μεταβαίνει στο **IDLE_STATE**, δίνεται τιμή στο final_output, σαν το αποτέλεσμα της ALU της προηγούμενης κατάστασης (POST_PROCESSING). Αν έχουμε υπερχείλιση σε κάποιο από τα στάδια, θέτουμε στο final_output την 32 bit τιμή που αντιστοιχεί στον δεκαδικό αριθμό -1. Επιπρόσθετα, όποτε συμβαίνει υπερχείλιση η το αποτέλεσμα μηδενίζεται, ενημερώνονται και οι αντίστοιχες έξοδοι που κρατούν το στάδιο στο οποίο συνέβη υπερχείλιση ή μηδενισμός του αποτελέσματος.

tb_nn.v

Για να ελέγξουμε ότι η υλοποίηση του νευρωνικού δικτύου λειτουργεί σωστά, δημιουργήσαμε ένα testbench module, στο οποίο αρχικοποιήσαμε την υλοποίηση του nn module. Στη συνέχεια, δημιουργούμε σήμα ρολογιού με

περίοδο 10ns και duty cycle 50%. Συντάσσουμε επιπλέον, ένα task με όνομα `nh_test_case` το οποίο δίνει στο δίκτυο τα παρακάτω τρία ζεύγη εισόδων:

- Τυχαία παραγόμενο ζεύγος με εύρος τιμών `[-4096, 4095]`.
- Τυχαία παραγόμενο ζεύγος με εύρος τιμών `[max_positive_number / 2, max_positive_number]` για έλεγχο υπερχείλισης θετικών αριθμών.
- Τυχαία παραγόμενο ζεύγος με εύρος τιμών `[max_negative_number, max_negative_number / 2]` για έλεγχο υπερχείλισης αρνητικών αριθμών.

Μετά από μία καθυστέρηση (ώστε το νευρωνικό δίκτυο να προλάβει να εξάγει το αποτέλεσμα), υπολογίζει την αναμενόμενη έξοδο από τη δοσμένη συνάρτηση της εκφώνησης (κάνουμε `include` την συνάρτηση `nh_model`) και συγκρίνει τα αποτελέσματα. Αν δεν ταιριάζουν, τυπώνει την χρονική στιγμή του λάθους, τις εισόδους και την πραγματική και αναμενόμενη έξοδο. Αν αντίθετα συμφωνούν, αυξάνουμε κατά 1 την τιμή ενός `global integer counter` που καταγράφει τις περιπτώσεις που έχουμε `pass`. Σε κάθε περίπτωση, αυξάνουμε κατά 1 την τιμή ενός πρόσθετου `integer counter` που καταγράφει τον αριθμό των συνολικών δοκιμών (`total cases`).

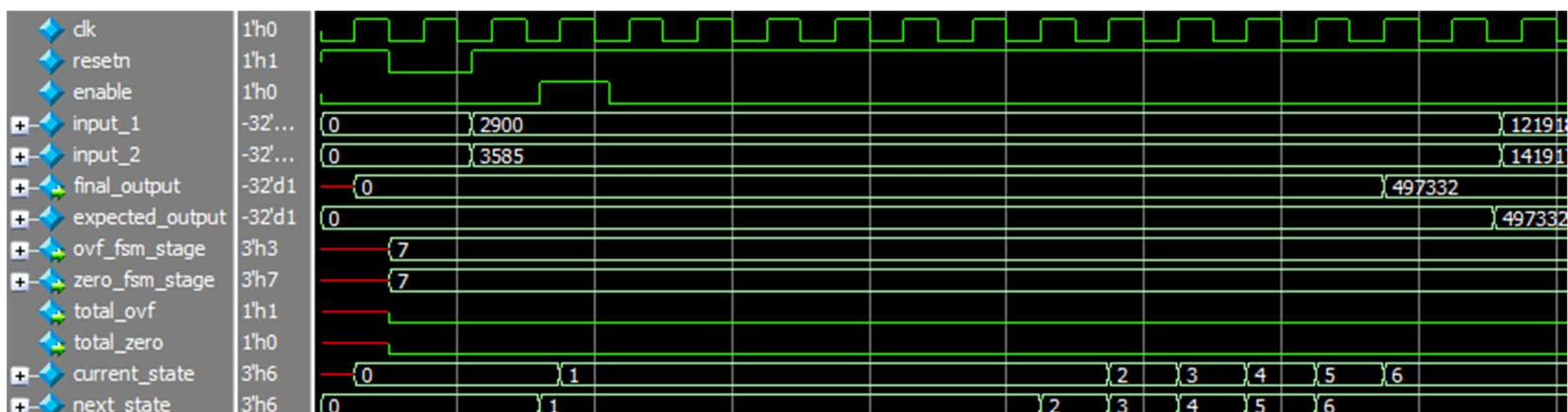
Στο `stimulus` του `testbench`, καλούμε το παραπάνω task για 100 επαναλήψεις και στο τέλος τυπώνουμε τους δύο counters [`pass test cases/total test cases`]. Εκτελώντας την προσομοίωση, με το κύκλωμα που υλοποιήσαμε, τυπώνεται το παρακάτω μήνυμα:

```
# Starting Simulation
# Number of PASS/Total Cases:      300/      300
# Simulation Ended
```

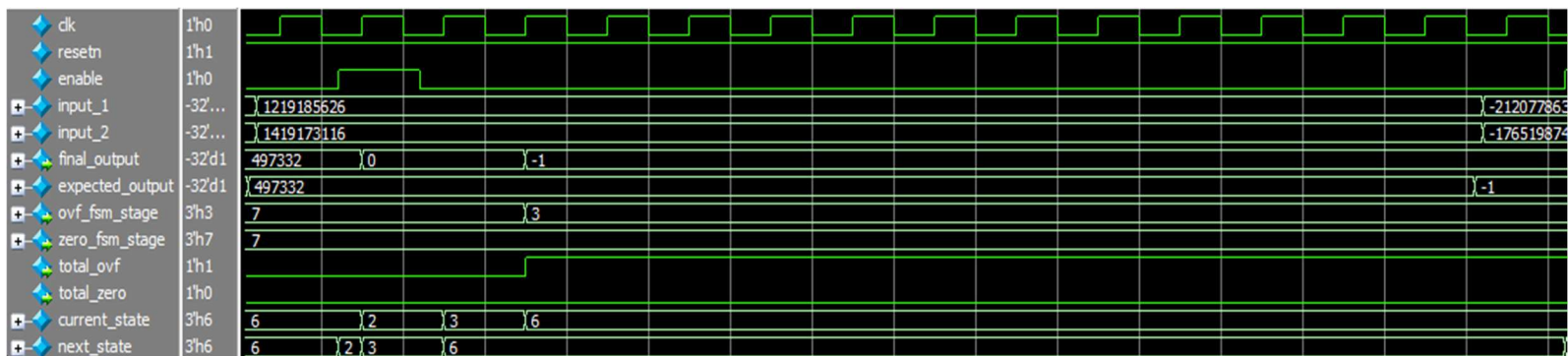
Παρατηρούμε ότι, η έξοδος της υλοποίησης μας, συμπίπτει με την αναμενόμενη έξοδο (`nh_model function`) και για τις 300 δοκιμές (3 τεστ για 100 επαναλήψεις).

Παρακάτω παραθέτουμε, τα διαγράμματα χρονισμού της προσομοίωσης, για την πρώτη επανάληψη, η οποία περιλαμβάνει ένα κανονικό τεστ και δύο τεστ για έλεγχο υπερχείλισης θετικών και αρνητικών αριθμών:

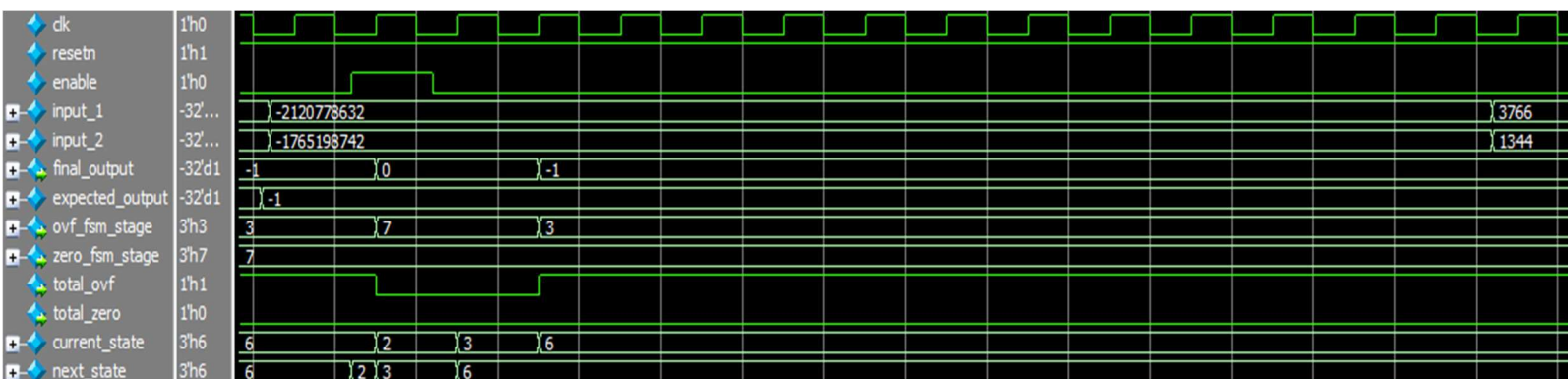
1-Διάγραμμα χρονισμού νευρωνικού δικτύου



2-Διάγραμμα χρονισμού νευρωνικού δικτύου (Υπερχείλιση θετικών αριθμών)



3-Διάγραμμα χρονισμού νευρωνικού δικτύου (Υπερχείλιση αρνητικών αριθμών)



Βλέπουμε ότι, για την πρώτη περίπτωση, το νευρωνικό δίκτυο ξεκινά στην DEACTIVATED_STATE, και όταν το enable πάρει την τιμή 1, ξεκινά να μεταβαίνει μεταξύ όλων των καταστάσεων, μέχρι να φτάσει στην IDLE_STATE (κατάσταση 6) και να ενημερώσει την έξοδο. Η τιμή της, ταιριάζει με αυτή του expected_output η οποία υπολογίζεται μερικούς κύκλους ρολογιού αργότερα. Επίσης οι έξοδοι total_onf και total_zero έχουν μηδενική τιμή, καθώς και τα onf_fsm_stage και zero_fsm_stage έχουν default τιμή (111), αφού δεν συνέβη υπερχείλιση, ούτε πήραμε μηδενικό αποτέλεσμα σε κανένα στάδιο του fsm. Στην δεύτερη και τρίτη περίπτωση, θέτουμε τις εισόδους σε μεγάλους θετικούς και αρνητικούς αριθμούς, ώστε να ελέγξουμε την συμπεριφορά του δικτύου σε υπερχείλιση. Πράγματι, όταν το enable γίνει 1 το fsm μεταβαίνει στην κατάσταση 2 (PRE_PROCESSING), αφού έχει φορτώσει ήδη μία φορά τα weights και biases από τη rom (Το fsm δεν μεταβαίνει ξανά στην κατάσταση LOADING_WEIGHTS_AND_BIASES σε όλη την υπόλοιπη διάρκεια της προσομοίωσης). Και στις δύο περιπτώσεις, όταν το fsm φτάνει στην κατάσταση 3 (INPUT_LAYER), ανιχνεύεται υπερχείλιση από τα mac_units. Έτσι, ενεργοποιείται το σήμα total_onf, και το onf_fsm_stage παίρνει την τιμή (3'd3 = 3'b011), δηλαδή το στάδιο στο οποίο συνέβη η υπερχείλιση. Το fsm μεταβαίνει απευθείας στην κατάσταση 6 (IDLE_STATE), και η έξοδος ενημερώνεται στην δεκαδική τιμή -1, η οποία ταιριάζει τελικά με το expected_output. Συνεπώς, το νευρωνικό δίκτυο έχει την αναμενόμενη συμπεριφορά.